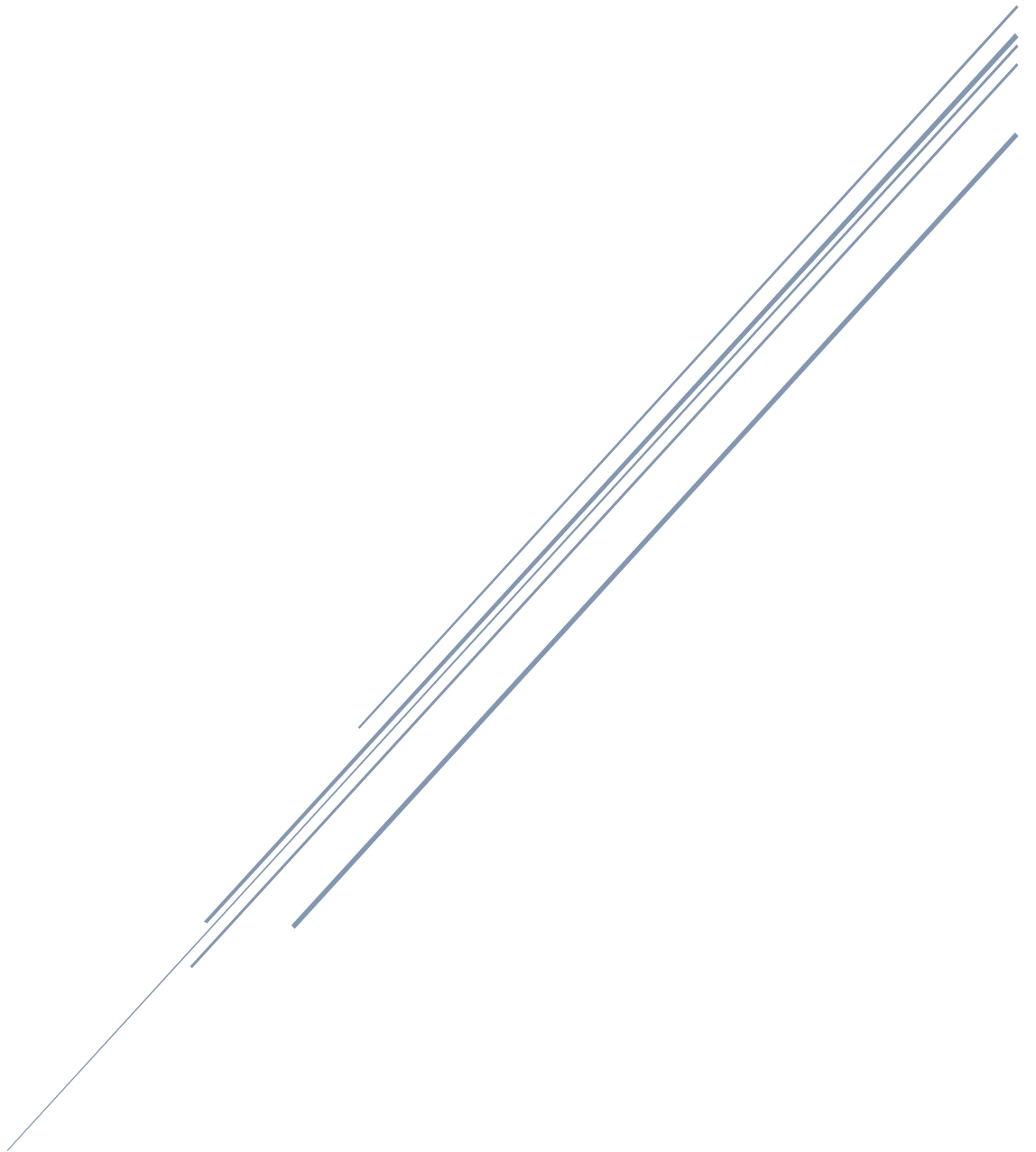


CREACIÓN DE UNA BASE DE DATOS

JOSE MANUEL ROMERO GARCÍA



ITACADEMY
DATA ANALYTICS

Tasca S4.01. Creación de base de datos

Datos del alumno

- Alumno: Jose Manuel Romero García
- Mentor: Yunier Cordova
- Curso: Especialización en Analítica de Datos

1. Introducción

El objetivo de esta tarea es construir una base de datos a partir de varios ficheros CSV y diseñar un modelo en estrella con una tabla de hechos y, como mínimo, cuatro tablas de dimensiones.

La base de datos creada se denomina **new_transactions** e incluye la carga de datos, la definición de claves primarias y foráneas, la creación de tablas derivadas y la resolución de distintos ejercicios mediante consultas SQL.

2. Descripción general del modelo

La estructura final sigue un esquema en estrella, donde la tabla de hechos **transactions** se sitúa en el centro y se relaciona con varias tablas de dimensiones.

Las tablas principales del modelo son:

- Tabla de hechos:
 - transactions
- Tablas de dimensiones:
 - users
 - companies
 - credit_cards
 - products
- Tablas de apoyo / derivada:
 - status_cards
- Tabla de transición:
 - products_transactions

3. Nivel 1 – Creación de la base de datos y modelo en estrella

3.1 Objetivos del Nivel 1

En el Nivel 1 se crean la base de datos y las tablas necesarias para construir un modelo en estrella. Se definen la tabla de hechos y las tablas de dimensiones, se ajustan tipos de datos básicos y se preparan las estructuras para la posterior carga de información.

3.2 Creación de la base de datos

Se crea la base de datos **new_transactions**, que contendrá todas las tablas de la solución.

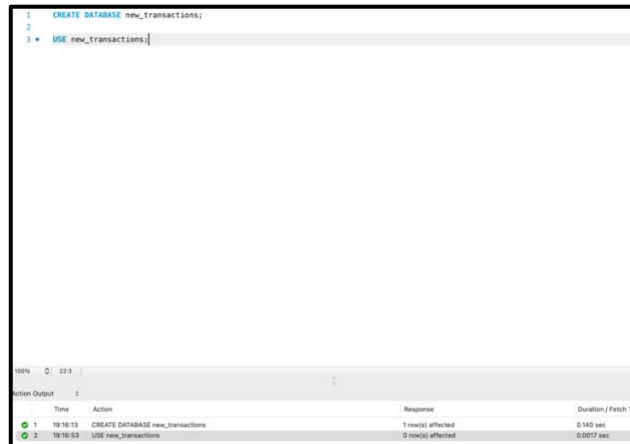


Figura 1. Creación de la base de datos.

3.3 Tablas de dimensiones

3.3.1 Tabla users

La tabla users unifica la información procedente de los ficheros american_users.csv y european_users.csv.

Campos principales: id, name, surname, phone, email, birth_date, country, city, postal_code, address.

La clave primaria es id (tipo INT), y el resto de campos se definen inicialmente como VARCHAR, ajustando la longitud según las necesidades. En el momento de la creación de las tablas no definiré las PK o FK.

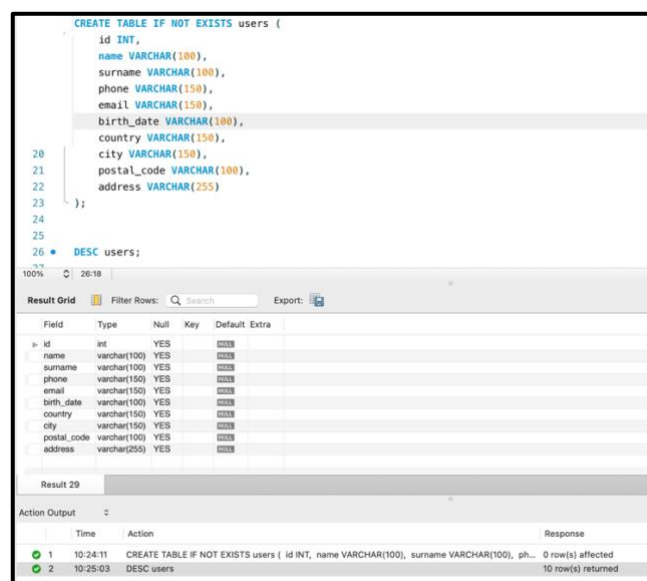


Figura 2. Creación de la tabla users.

3.3.2 Tabla companies

La **tabla companies** contiene datos de las empresas: company_id, company_name, phone, email, country, website. La clave primaria es company_id y los campos se definen como VARCHAR, optimizando especialmente phone y email.

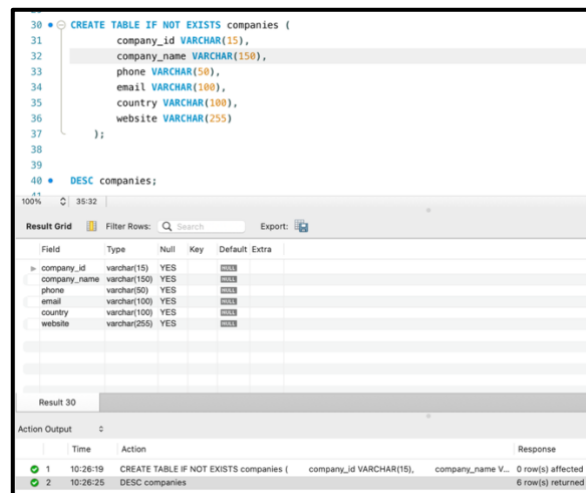


Figura 3. Creación de la tabla companies.

3.3.3 Tabla credit_cards

La tabla **credit_cards** almacena información de tarjetas de crédito: id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date. La clave primaria es id.

El campo user_id se define con el mismo tipo que users.id para permitir la relación entre ambas tablas, en este caso INT.

A los campos pin y cvv se les da un tipo de dato VARCHAR(4) por eficiencia, y expiring_date se crea como VARCHAR(100). Evito el tipo de dato (INT) para cvv, por el problema que se puede generar si empezase con cero, por ejemplo 012, si lo guardo en (INT) quedaría como 12. Posteriormente, se realizarán los cambios necesarios.

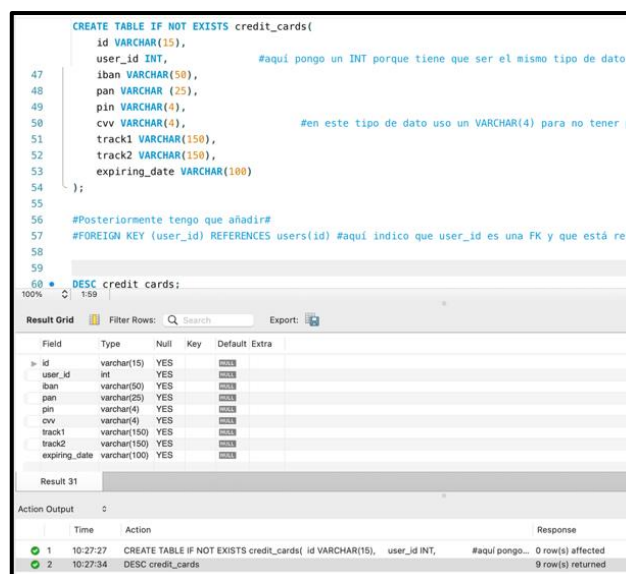


Figura 4. Creación de la tabla credit_cards

3.3.4 Tabla products

La **tabla products** contiene datos de productos: id, product_name, price, colour, weight, warehouse_id. La clave primaria es id con un tipo de dato INT. El campo weight se define como DECIMAL(4,1) y colour como VARCHAR(20). El campo price se mantiene inicialmente como VARCHAR para decidir posteriormente su conversión a un tipo numérico, ya que si se tuvieran que hacer cálculos con este campo necesitaríamos cambiar el tipo de valor.

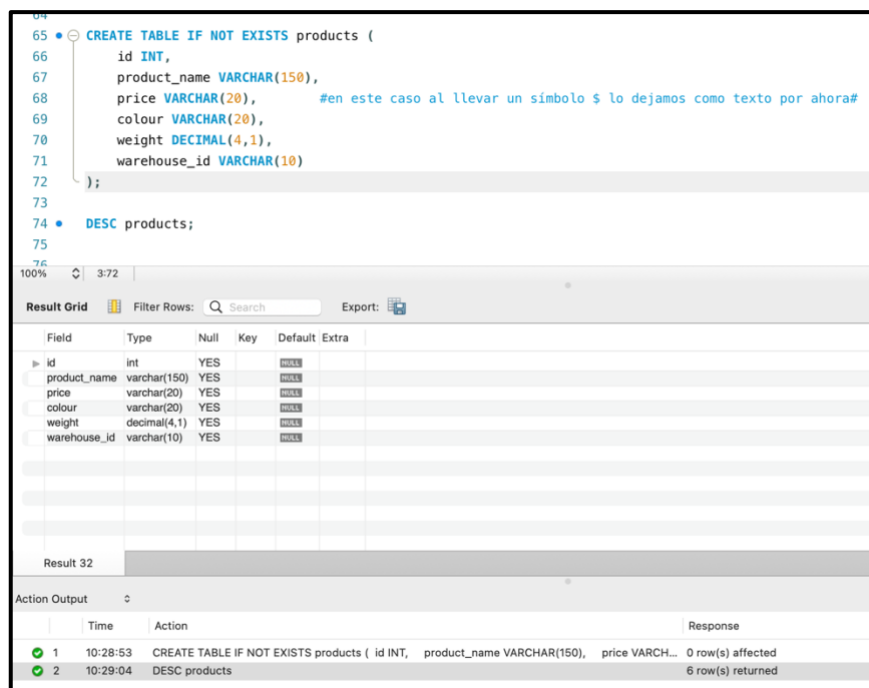


Figura 5. Creación de la tabla products.

3.4 Tabla de hechos transactions

La **tabla transactions** almacena el detalle de las operaciones realizadas: Campos: id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude. Inicialmente se establecen todos los tipos de datos como VARCHAR. Posteriormente tendré que ajustar el tipo de datos a las necesidades y establecer la clave primaria y las claves foráneas. Esto se realizará una vez se carguen los datos

- id es la clave primaria.
- Claves foráneas:
 - card_id → credit_cards.id
 - business_id → companies.company_id
 - user_id → users.id
- Ajustes clave de tipos de datos:
 - timestamp → TIMESTAMP

- amount → DECIMAL(10,2)
- declined → BOOLEAN

```

79 • CREATE TABLE IF NOT EXISTS transactions (
80     id VARCHAR(255),
81     card_id VARCHAR(15),
82     business_id VARCHAR(15),
83     timestamp VARCHAR(25),
84     amount VARCHAR(50),
85     declined VARCHAR(10),
86     product_ids VARCHAR(25),
87     user_id VARCHAR(25),
88     lat VARCHAR(50),
89     longitude VARCHAR(50) #FOREIGN KEY (card_id) REFERENCES credit_cards(id)
90                           #FOREIGN KEY (business_id) REFERENCES companies(company_id)
91                           #FOREIGN KEY (user_id) REFERENCES users(id)
92 );
93
94 • DESC transactions;
95
100% 19:34

```

Result Grid

Field	Type	Null	Key	Default	Extra
id	varchar(255)	YES			
card_id	varchar(15)	YES			
business_id	varchar(15)	YES			
timestamp	varchar(25)	YES			
amount	varchar(50)	YES			
declined	varchar(10)	YES			
product_ids	varchar(25)	YES			
user_id	varchar(25)	YES			
lat	varchar(50)	YES			
longitude	varchar(50)	YES			

Result 33

Action Output

	Time	Action	Response
1	10:30:14	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255), card_id VARCHAR(15), busin...	0 row(s) affected
2	10:30:23	DESC transactions	10 row(s) returned

Figura 6. Creación de la tabla transactions.

3.5 Configuración y carga de datos

Para permitir el uso de LOAD DATA LOCAL INFILE, se habilita la opción local_infile = 1 y se configura la conexión con la opción OPT_LOCAL_INFILE=1.

A continuación, se cargan los datos de los ficheros CSV en el siguiente orden:

1. american_users y european_users en users.
2. Datos en companies, products y credit_cards.
3. Datos en transactions como tabla de hechos.

Tras cada carga se ejecutan consultas SELECT para comprobar la correcta importación.

```

97
98 • SET GLOBAL local_infile = 1;
99
100
100% 5:59

```

Action Output

	Time	Action	Response	Duration / Fetch
1	22:58:04	SET GLOBAL local_infile = 1	0 row(s) affected	0.0099 sec

Figura 7. Activación local_infile.

Comprobación de que está activado:

SHOW GLOBAL VARIABLES LIKE 'local_infile';

Variable_name	Value
local_infile	ON

	Time	Action
1	23:21:47	SHOW GLOBAL VARIABLES LIKE 'local_infile'

Figura 8. Comprobación local_infile está ON.

Inicio la carga de los datos en la **tabla users**:

```

110 • LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/Sprint_4/american_users.csv'
111 INTO TABLE users
112 FIELDS TERMINATED BY ','
113 ENCLOSED BY '"'
114 LINES TERMINATED BY '\n'
115 IGNORE 1 LINES;
116
117 #({id, name, surname, phone, email, birth_date, country, city, postal_code, address})#
118
119 #Siguiendo estos pasos no consigo cargar la tabla. Sale error 2068#
120
121 #Sigo las instrucciones para averiguar en qué carpeta se puede copiar#
122
123 • SHOW VARIABLES LIKE 'secure_file_priv';
124
125 • SELECT VERSION();
126

```


	Time	Action	Response	Duration / Fetch Time
1	11:51:45	SHOW GLOBAL VARIABLES LIKE 'local_infile'	1 row(s) returned	0.0018 sec / 0.00001...
2	11:51:54	LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/...	1010 row(s) affected Records: 1010 Deleted: 0 Skipp...	0.041 sec
3	11:52:16	SELECT * FROM new_transactions.users	1010 row(s) returned	0.0015 sec / 0.0023 s...

Figura 9. Carga de los datos del documento american_users.csv.

He decidido cargar los datos de los documentos american_users.csv y european_users.csv en la misma tabla. Finalmente, no he creído necesario crear una nueva columna con los datos que hacen referencia al continente, que al juntar ambos documentos se pierde esta información. La razón es porque para este ejercicio no se solicita y siempre se puede crear posteriormente.



Figura 10. Carga de los datos del documento european_users.csv.

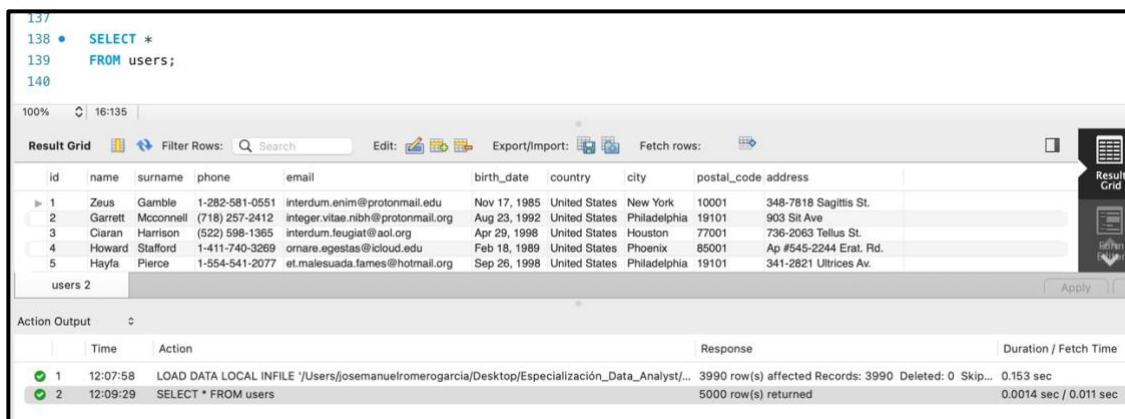


Figura 11. Comprobación de la carga de los datos en la tabla users.

Carga y comprobación de los datos en la **tabla companies**:

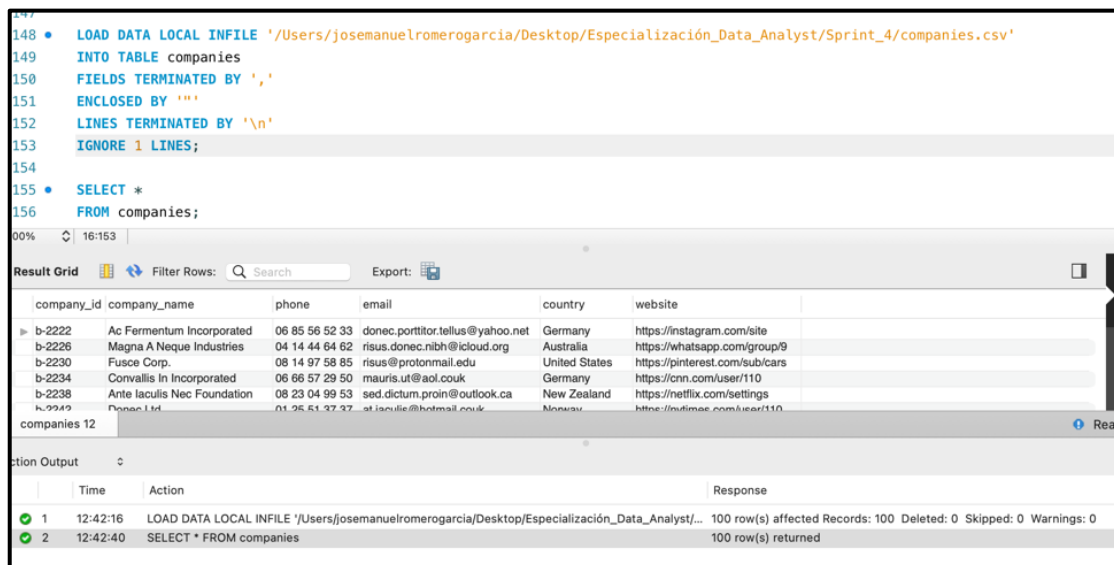


Figura 12. Carga de los datos del documento companies.csv y comprobación de la carga de los datos.

Carga y comprobación de los datos en la **tabla products**:

```

158 #Carga los datos en la tabla products#
159
160 • LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/Sprint_4/products.csv'
161 INTO TABLE products
162 FIELDS TERMINATED BY ','
163 ENCLOSED BY '"'
164 LINES TERMINATED BY '\n'
165 IGNORE 1 LINES;
166
167 • SELECT *
168 FROM products;
169
170
171

```

Result Grid

id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	\$161.11	#7c7c7c	1.0	WH-4
2	Tarly Stark	\$9.24	#919191	2.0	WH-3
3	duel tourney Lannister	\$171.13	#d8d8d8	1.5	WH-2
4	warden south duel	\$71.89	#111111	3.0	WH-1
5	skyscraper ewok	\$171.22	#d8d8d8	3.2	WH-0
6	direwolf ewok	\$156.60	#d8d8d8	0.8	WH-1

Action Output

Time	Action	Response
12:46:27	LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
12:46:49	SELECT * FROM products	100 row(s) returned

Figura 13. Carga de los datos del documento products.csv y comprobación de la carga de los datos.

Carga y comprobación de los datos en la **tabla transactions** y posterior comprobación: Es importante detallar el cambio en *fields terminated by* debido a que los valores están separados por ';' y no por ',' como en los documentos anteriores.

```

182
183 #Carga los datos en la tabla transactions#
184
185 • LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/Sprint_4/transactions.csv'
186 INTO TABLE transactions
187 FIELDS TERMINATED BY ';' #Para cargar estos datos tengo que variar la ',' por ';'
188 LINES TERMINATED BY '\n' #También quito '"' porque en estos datos no hay#
189 IGNORE 1 LINES;
190
191

```

Action Output

Time	Action	Response
16:25:17	LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Figura 14. Carga de los datos del documento transactions.csv en la tabla transactions.

```

192 • SELECT *
193 FROM transactions;
194
195

```

Result Grid

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
CDDA7E40-544D-47BB-A4ED-671DD8A850D9	CcS-6894	b-2466	2018-12-12 08:05:17	161.88	0	75, 73, 98	2313	59.62050974356148	16.559977155728436
09456357-8E9B-475A-8257-67A023699964	CcS-5135	b-2342	2024-05-20 22:49:08	171.13	0	3	554	45.76458841901318	4.843056518287656
C47C7C84-C174-4973-A76B-825A9DB582A4	CcS-8415	b-2250	2018-11-04 23:16:18	497.29	0	92, 85, 36, 23	3834	52.06849608409912	4.301099382555438
2FB526AA-3844-4DDF-AC09-2EBCC255EE78	CcS-6553	b-2610	2022-06-17 09:17:25	344.15	0	5, 27	1972	39.47598513179192	-0.3764194439184784
0D877407-B85E-40D2-9229-6BD6F2B8DFA0	CcS-7678	b-2454	2020-11-29 07:29:16	435.1	0	36, 32, 82, 62	3087	51.43656985032189	5.478525648707596
A0BD09EE-7199-410F-9CB5-3205508FA02F	CcS-6416	b-2438	2021-05-24 22:28:26	364.11	0	92, 64	1835	51.49519189047599	18.88912480174568
29322BF7-84F4-4A6C-9FC0-C086B8BD8DCD	CcS-5036	b-2222	2022-05-19 04:50:50	678.61	0	6, 12, 100, 22, 23	455	51.609077736775525	19.160038756363225
3F614EB9-1EEC-4FB7-8B8B-ABCD6C519EED	CcS-8216	b-2526	2019-08-18 21:09:25	207.52	0	69, 97, 31	3635	33.45816749565129	-112.08289579719126
133534F1-81E5-4688-B4AF-54E516504E9E	CcS-7160	b-2614	2021-12-23 23:58:33	368.64	0	51, 3, 68	2579	60.481217507242114	18.810024018503643
185D9FC6-9540-4E4F-B38D-395B26C467AC	CcS-5819	b-2550	2019-03-23 19:42:15	57.25	0	69	1238	42.14540699947335	12.21245353636445

Action Output

Time	Action	Response
16:25:17	LOAD DATA LOCAL INFILE '/Users/josemanuelromerogarcia/Desktop/Especialización_Data_Analyst/...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
16:25:49	SELECT * FROM transactions	100000 row(s) returned

Figura 15. Comprobación de que los datos se han cargado correctamente en la tabla transactions.

Una vez cargados los datos en las distintas tablas se inicia el proceso para cambiar los tipos de valor y establecer las relaciones entre las tablas de dimensiones y la tabla de hechos.

Cambios que se realizan en la **tabla users**:

- Se establece como PRIMARY KEY el campo users.id



Figura 16. Creación de la PK en la tabla users.

- Se modifica el tipo de valor de phone, pasa de VARCHAR(150) a VARCHAR(25):

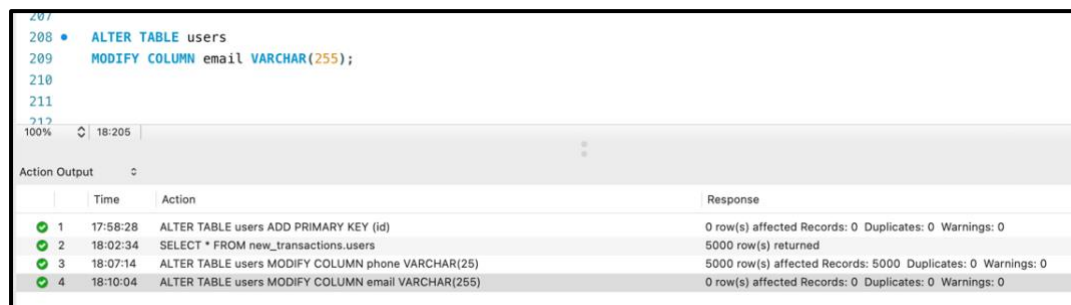


Figura 17. Modificación del tipo de valor del campo phone.

- Se modifica el tipo de valor de email pasa de VARCHAR(150) a VARCHAR(255):

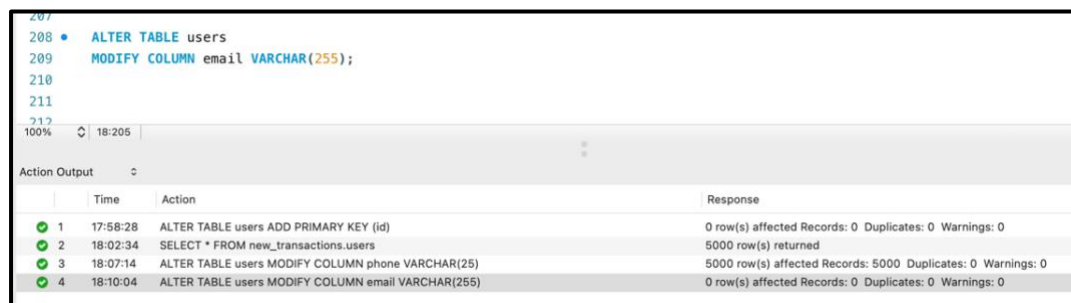


Figura 18. Modificación del tipo de valor del campo email.

- Se modifica el tipo de valor de birth_date de VARCHAR(100) a VARCHAR(15):



Figura 19. Modificación del tipo de valor del campo birth_date.

- Se modifica el tipo de valor de country de VARCHAR(150) A VARCHAR(100):



Figura 20. Modificación del tipo de valor del campo country.

- Se modifica el tipo de valor de postal_code de VARCHAR(100) a VARCHAR(20):

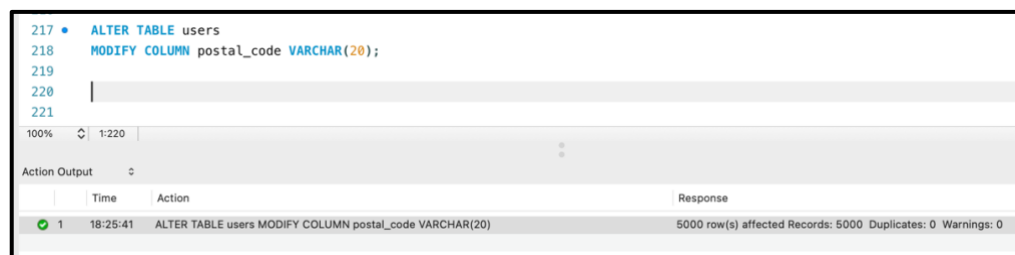


Figura 21. Modificación del tipo de valor del campo postal_code.

Comprobación y visualización de los cambios realizados en la **tabla users**:

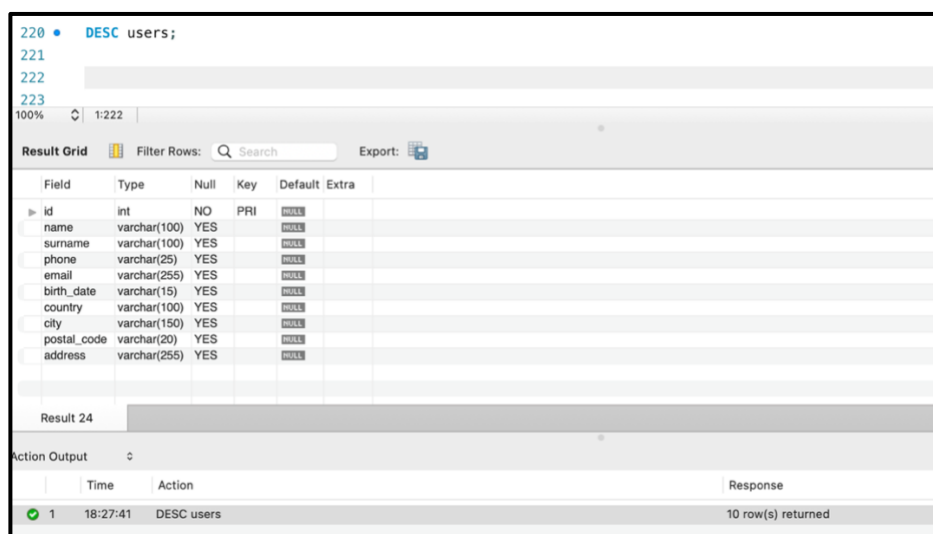


Figura 22. Comprobación de los cambios realizados en la tabla users.

Cambios que se realizan en la **tabla companies**:

- Se añade la PK de la tabla en el campo company_id:



Figura 23. Creación de la PK de la tabla companies.

- Se modifica el tipo de valor del campo phone de VARCHAR(50) a VARCHAR(25):

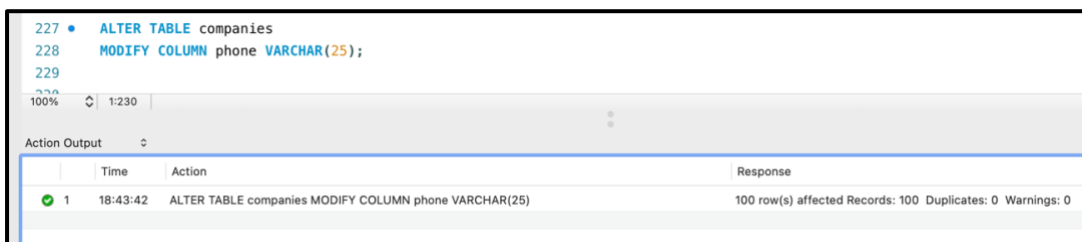


Figura 24. Modificación del tipo de valor del campo phone.

- Se modifica el tipo de valor del campo email de VARCHAR(100) a VARCHAR(255):

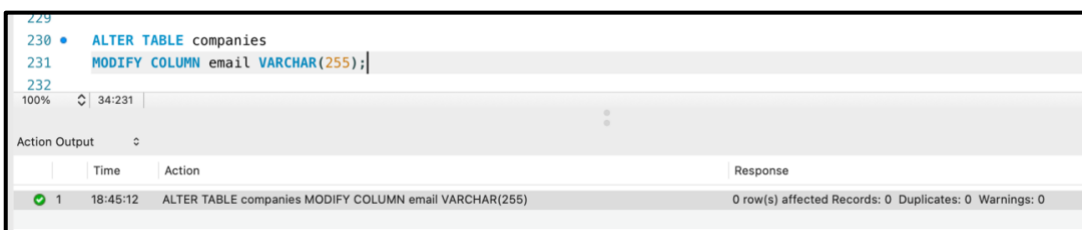


Figura 25. Modificación del tipo de valor del campo email.

La **tabla companies** queda de la siguiente manera:

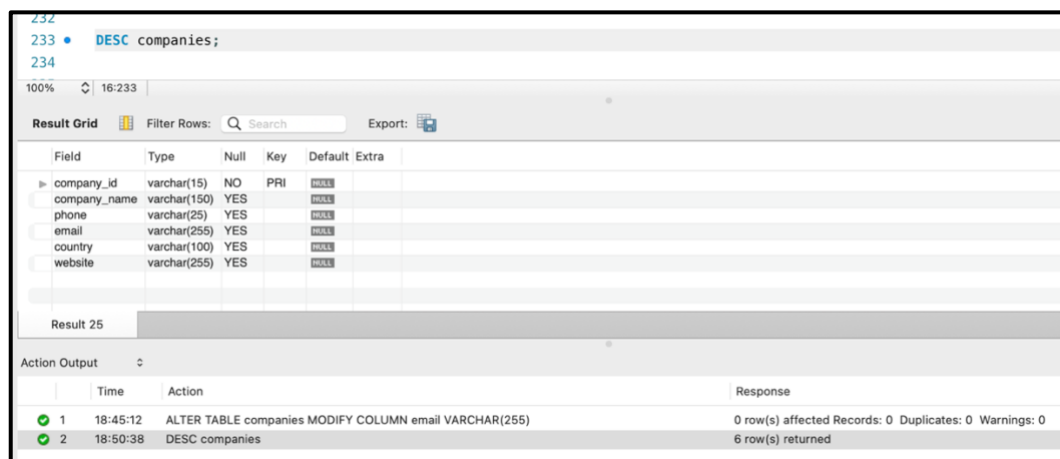


Figura 26. Comprobación de los cambios realizados en la tabla companies

Cambios que se realizan en la **tabla credit_cards**:

- Se establece la PK en el campo id:

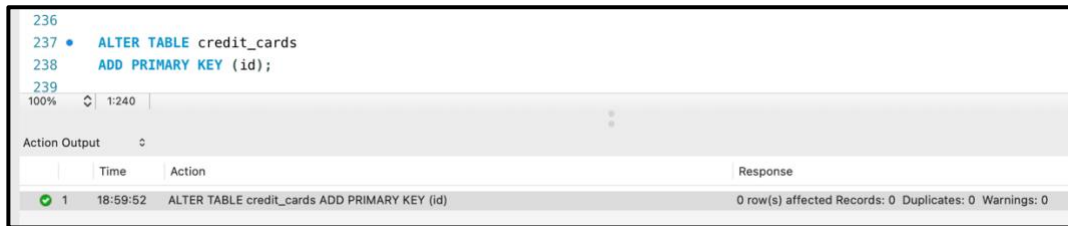


Figura 27. Creación de la PK de la tabla credit_cards.

- Se establece como FK el campo user_id que me permitirá establecer una relación con la tabla users.

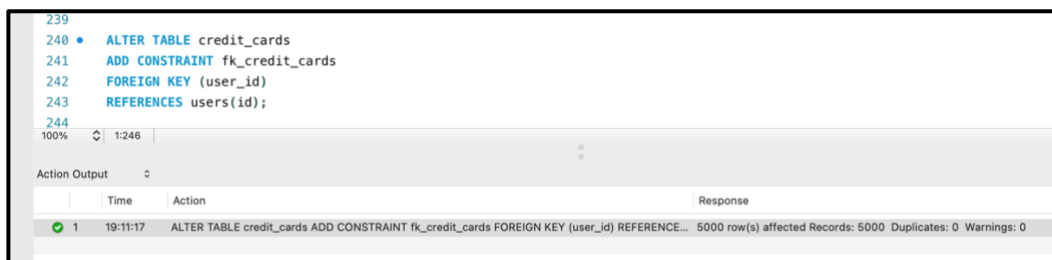


Figura 28. Creación de la FK de la tabla credit_cards.

- Se modifican los tipos de valor de los campos pin y cvv que los paso de VARCHAR(4) a CHAR(4), por un tema de eficiencia y evitar problemas que se pueden presentar cuando se utilizan ceros. Esto se hace ya que ambos tipos de datos tiene una longitud fija.



Figura 29. Modificación del tipo de valor del campo pin.

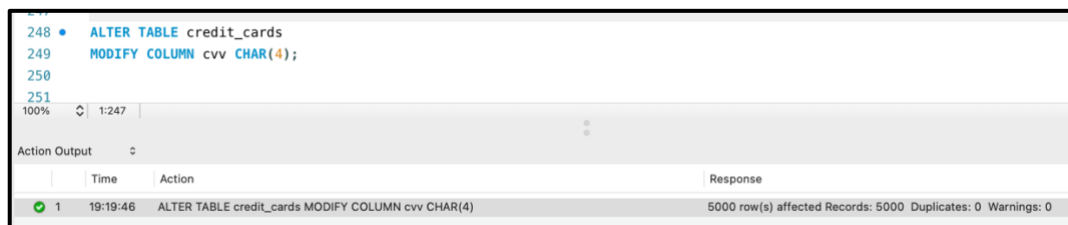
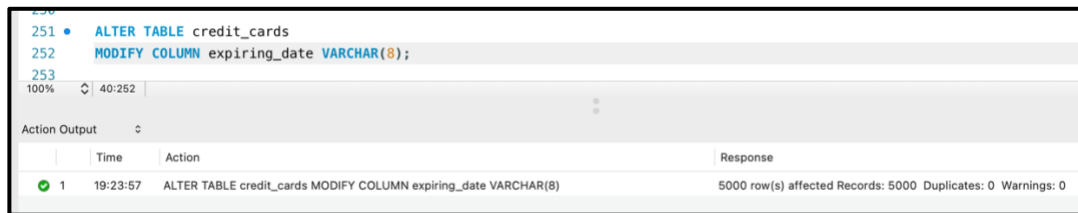
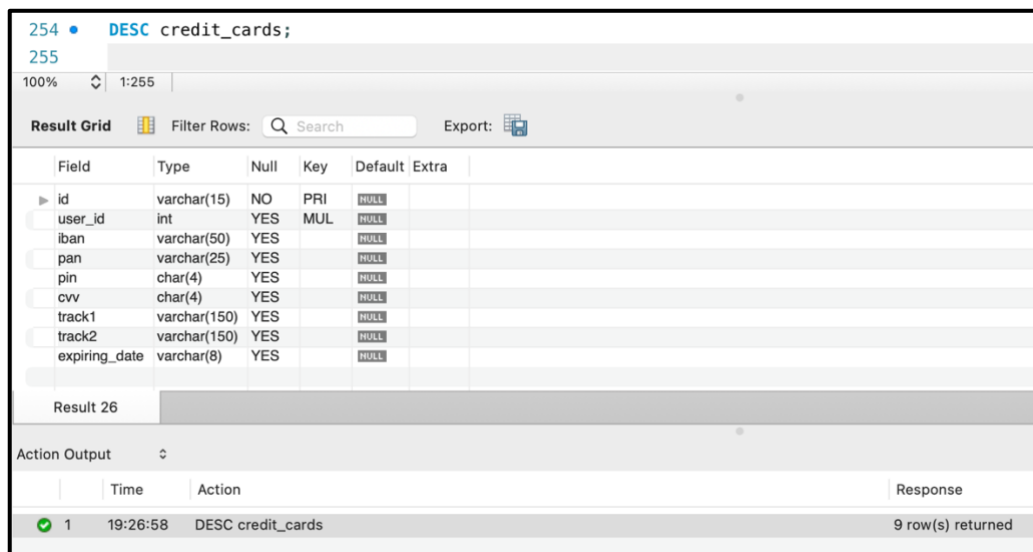


Figura 30. Modificación del tipo de valor del campo cvv.

- Se modifica el tipo de valor del campo `expiring_date` de `VARCHAR(100)` a `VARCHAR(8)`:

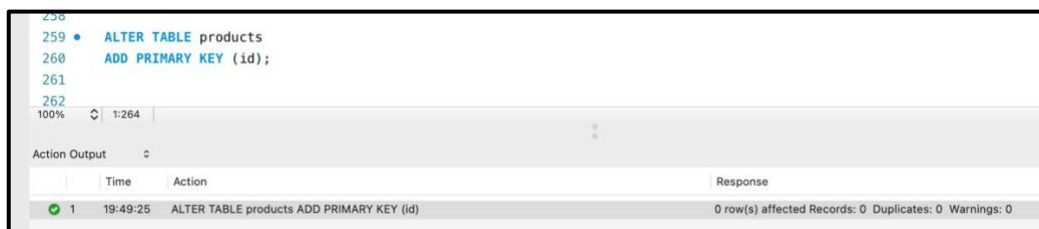
Figura 31. Modificación del tipo de valor del campo `expiring_date`.

Tras los cambios realizados en la **tabla credit_cards**, la tabla tendrá estos tipos de valores:

Figura 32. Comprobación de los cambios realizados en la tabla `credit_cards`.

Cambios que se realizan en la **tabla products**.

- Se establece el campo (`id`) de la tabla `products` como PK.

Figura 33. Creación de la PK de la tabla `products`.

- Se modifica el tipo de valor de colour de VARCHAR(20) a CHAR(7), debido a que los datos tiene una longitud fija:



Figura 34. Modificación del tipo de valor del campo colour.

De momento no voy a tocar *price*, lo dejo como lo tengo hasta más tarde para ver si lo convierto en decimal y qué hago con el símbolo del dólar. La **tabla products** me quedaría de la siguiente forma según los tipos de datos:

```

264
265 • DESC products;
266
267
100% 1:264

```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		
product_name	varchar(150)	YES			
price	varchar(20)	YES			
colour	char(7)	YES			
weight	decimal(4,1)	YES			
warehouse_id	varchar(10)	YES			

Result 28

	Time	Action	Response
1	19:58:58	DESC products	6 row(s) returned

Figura 35. Comprobación de los cambios en la tabla products.

Cambios en la tabla de hechos **transactions**.

- Se establece el campo (id) como PK de la **tabla transactions**.

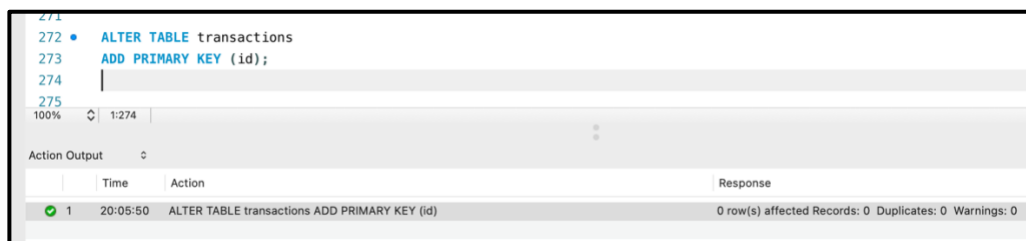


Figura 36. Determinación de la Primary Key de la tabla transactions.

- Se determinan las FK que la **tabla transactions** tiene con las otras tablas: en la **tabla credit_cards** el campo (id) se relaciona con la **tabla transactions** mediante el campo (card_id). Los dos campos tienen como tipo de valor un VARCHAR(15).

```

274
275 • ALTER TABLE transactions
276   ADD CONSTRAINT fk_transactions_credit_cards
277   FOREIGN KEY (card_id)
278   REFERENCES credit_cards(id); #los dos campos tiene el mismo tipo de dato VARCHAR(15)#
279
280
281
100% 91:278

```

	Time	Action	Response
1	20:11:34	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (card_id)...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Figura 37. Determinación de la FK en la tabla transactions que se relaciona con la tabla credit_cards.

- Determinamos otra FK en la **tabla transactions**. Se establece la relación con la **tabla companies** a través de los campos (company_id) y (business_id), en la **tabla transactions**. Ambos campos tienen como tipo de valor VARCHAR(15).

```

281 • ALTER TABLE transactions
282   ADD CONSTRAINT fk_transactions_companies
283   FOREIGN KEY (business_id)
284   REFERENCES companies(company_id);
285
100% 25:281

```

	Time	Action	Response
1	20:16:39	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (business_id)...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Figura 38. Determinación de la FK de la tabla transactions que establece la relación con la tabla companies.

- Se establece la FK con la **tabla users**. Primero tengo que cambiar el tipo de dato en la **tabla transactions**. El tipo de dato de (user_id) en la **tabla transactions** debe ser un valor INT como en la **tabla users**.

```

283
286 • ALTER TABLE transactions
287   MODIFY COLUMN user_id INT;
288
289
290 • ALTER TABLE transactions
291   ADD CONSTRAINT fk_transactions_users
292   FOREIGN KEY (user_id)
293   REFERENCES users(id);
294
100% 23:293

```

	Time	Action	Response
1	20:22:42	ALTER TABLE transactions MODIFY COLUMN user_id INT	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	20:22:51	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_users FOREIGN KEY (user_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Figura 39. Determinación de la FK de la tabla transactions que la relaciona con la tabla users.

- La FK de la **tabla transactions** que establecería la relación con la **tabla products** no la establecemos, de momento.
- Se modifica el tipo de valor del campo timestamp de VARCHAR(25) a TIMESTAMP.

```

297 • ALTER TABLE transactions
298   MODIFY COLUMN timestamp TIMESTAMP;
299
300
100% 49:295

```

	Time	Action	Response
1	10:53:48	ALTER TABLE transactions MODIFY COLUMN timestamp TIMESTAMP	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Figura 40. Modificación del tipo de valor del campo timestamp.

- Se modifica el tipo de valor del campo amount de VARCHAR(50) a DECIMAL(10,2), de forma que permita realizar cálculos con el dato, mantenerlo como VARCHAR aunque permite hacer cálculos podría dar problemas.

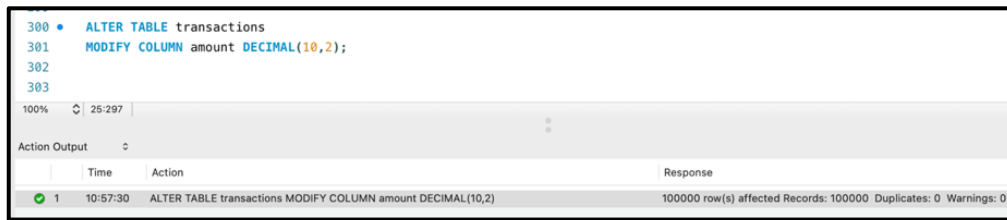


Figura 41. Modificación del tipo de valor del campo amount.

- Se modifica el tipo de valor del campo declined de VARCHAR(10) a BOOLEAN. MySQL guarda el dato como TINYINT(1).

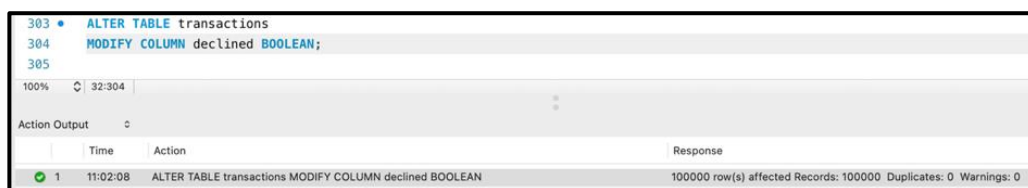


Figura 42. Modificación del tipo de valor del campo declined.

La **tabla transactions** queda de la siguiente manera:

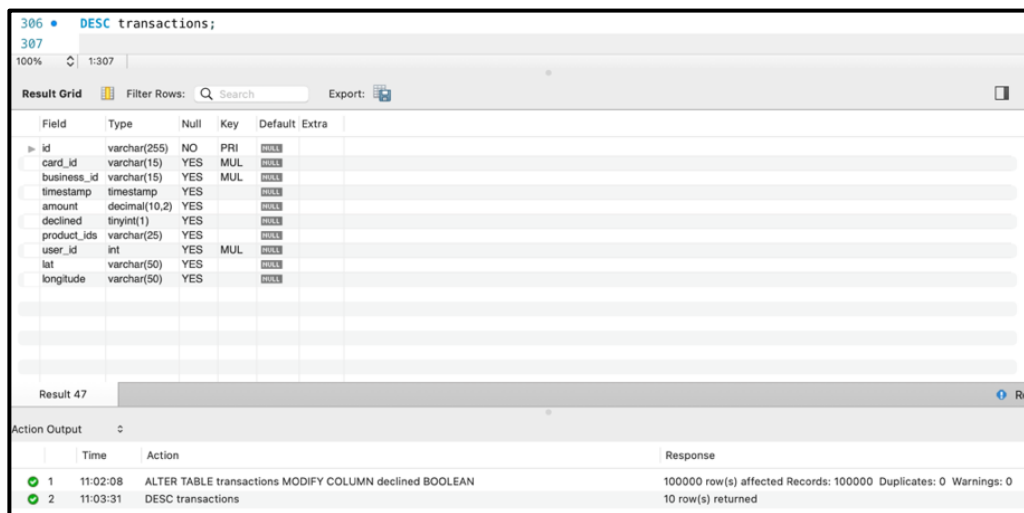


Figura 43. Comprobación de los cambios en la tabla transactions.

Una vez realizados todos los cambios compruebo el diagrama resultante para ver si se ajusta a lo pedido en el ejercicio, un diagrama con un esquema de estrella.

El diagrama obtenido es el siguiente:

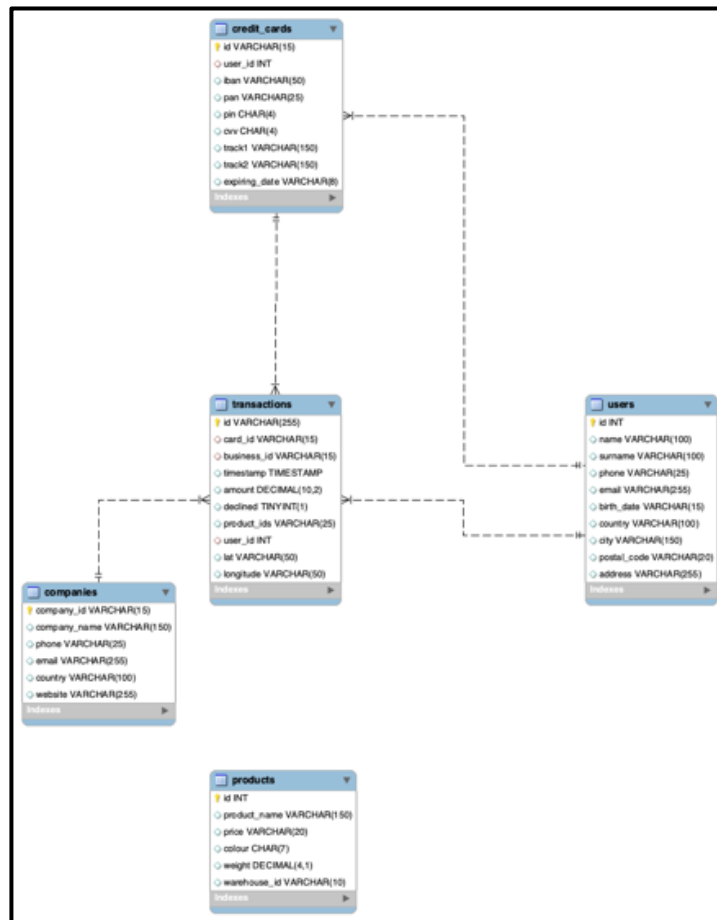


Figura 44. Diagrama resultante después de las primeras modificaciones.

Como el ejercicio me especifica que debo tener un modelo de estrella, decido eliminar la relación que se establece entre la tabla users y la tabla credit_cards.

```

313 • ALTER TABLE credit_cards
314 DROP FOREIGN KEY fk_credit_cards;
315
316
100% 1:315
Action Output
Time Action Response
1 11:30:22 ALTER TABLE credit_cards DROP FOREIGN KEY fk_credit_cards 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

```

Figura 45. Eliminación de la FK que establece la relación entre las tablas users y credit_cards.

El diagrama que se obtiene en este momento del Sprint 4, es el siguiente, un modelo de estrella:

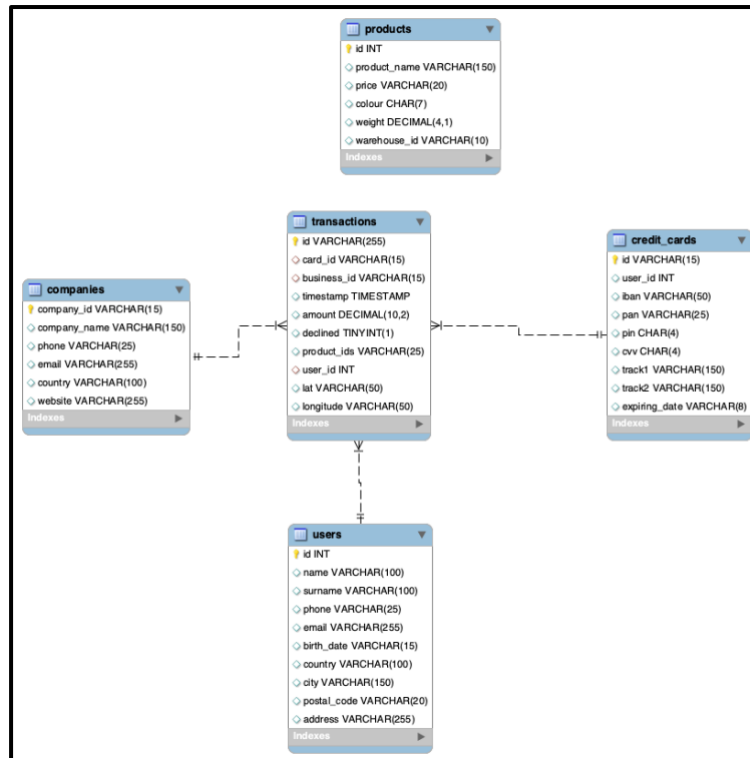


Figura 46. Diagrama nuevo, modelo de estrella.

3.6 Ejercicios del Nivel 1

Ejercicio 1 – Mostrar los usuarios con más de 80 transacciones mediante una subconsulta, se deben utilizar como mínimo 2 tablas.

```

318 SELECT u.name AS name,
319        u.surname AS surname,
320        u.id
321 FROM users u
322 WHERE u.id IN (SELECT t.user_id
323               FROM transactions t
324               WHERE t.declined = 0
325               GROUP BY t.user_id
326               HAVING COUNT(DISTINCT(t.id)) > 80)
327 ORDER BY u.id;
  
```

Result Grid

name	surname	id
Molly	Gilliam	185
Diego	Hecru	289
Bmyr	Astue	318

users 2

Action Output

Time	Action	Response
10:15:00	SELECT u.name AS name, u.surname AS surname, u.id FROM users u WHERE u.id IN (SELEC...	3 row(s) returned

Figura 47. Cantidad de usuarios con más de 80 transacciones.

Ejercicio 2 – Media de amount por IBAN de “Donec Ltd”.

Se construye una consulta que muestra la media de amount por IBAN de las tarjetas de crédito de la compañía “Donec Ltd”, utilizando al menos dos tablas.

```

335 SELECT ROUND(AVG(t.amount),2) AS Cantidad_media,
336         cc.iban AS Número_Iban,
337         cc.id AS Identificador_targeta,
338         c.company_name AS Compañía
339 FROM credit_cards cc
340 JOIN transactions t ON t.card_id = cc.id
341 JOIN companies c ON c.company_id = t.business_id
342 WHERE c.company_name = 'Donec Ltd' AND t.declined = 0
343 GROUP BY cc.iban, cc.id, c.company_name
344 ORDER BY AVG(t.amount) DESC;

```

Cantidad_media	Número_Iban	Identificador_targeta	Compañía
680.69	XX383017813919620199366352	CcS-7935	Donec Ltd
680.01	XX637706357397570394973913	CcS-6398	Donec Ltd
645.46	XX971393971465292202312259	CcS-6374	Donec Ltd
628.89	XX171847116928892375969307	CcS-7851	Donec Ltd
608.68	XX225424639818542406223575	CcS-6502	Donec Ltd
607.29	XX748890729057195711766071	CcS-5121	Donec Ltd
605.41	TN9614563570657381893122	CcU-3239	Donec Ltd
605.36	XX481908034037364242591185	CcS-6646	Donec Ltd
597.19	XX19467551973925635753508	CcS-7186	Donec Ltd
594.26	XX215962766061967195493437	CcS-6435	Donec Ltd
591.61	XX44932320826890721001443	CcS-9131	Donec Ltd
570.09	XX535185492735704229474237	CcS-5872	Donec Ltd
566.38	CH9552373968796160224	CcU-4177	Donec Ltd
561.80	XX347605377125637880303131	CcS-8326	Donec Ltd
543.42	XX688471446697921912860304	CcS-8532	Donec Ltd
542.00	XX605533964582458704105956	CcS-8626	Donec Ltd

Result 1

Action Output

Time	Action	Response
10:23:01	SELECT ROUND(AVG(t.amount),2) AS Cantidad_media, cc.iban AS Número_Iban, cc.id AS I...	370 row(s) returned

Figura 48. Media de amount por IBAN de “Donec Ltd”.

4. Nivel 2 – Tablas derivadas y estado de tarjetas

4.1 Objetivos del Nivel 2

En el Nivel 2 se crea una tabla derivada que permite analizar el estado de las tarjetas (activas o inactivas) en función de sus últimas transacciones. Se utilizan funciones de ventana y expresiones de tabla comunes (CTE) para implementar la lógica.

4.2 Creación de la tabla status_cards

La tabla status_cards identifica si cada tarjeta está activa o inactiva, según si las tres últimas transacciones han sido declinadas.

Pasos realizados:

1. Creación de la tabla status_cards, donde card_id actúa como clave primaria.
2. Creación de una CTE last_transactions utilizando la función de ventana ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) para numerar las transacciones de cada tarjeta por orden de fecha.

3. A partir de la CTE, se aplica un CASE que marca una tarjeta como inactiva cuando sus tres últimas transacciones están declinadas; en caso contrario, se marca como activa.
4. Inserción de los resultados en status_cards.
5. Cálculo del número total de tarjetas activas.
6. Establecimiento de la relación entre status_cards y credit_cards.



Figura 49. Creación de la tabla status_cards.

Una vez creada la tabla necesito que el contenido me devuelva aquellas tarjetas que están activas o inactivas según las últimas 3 transacciones. Si las tres últimas transacciones han sido declinadas entonces la tarjeta se muestra como inactiva.

Para ello utilizaré la función ventana ROW_NUMBER() OVER PARTITION BY que asigna un número consecutivo a cada fila según un orden definido, en este caso para cada valor de (card_id) y posteriormente lo ordeno según la fecha en orden descendente.

Con esto construyo una CTE que llamo last_transactions sobre la cual luego puede hacer una consulta que me devuelve las tarjetas que están activas o inactivas. Para ello aplicaré un CASE.



Figura 50. CTE y Función ventana.

```

INSERT INTO status_cards (card_id, status) #esto lo agrego al final del proceso cuando ya esta todo creado para no repetir código
#entonces procedo a hacer el INSERT INTO

WITH last_transactions AS (
  SELECT
    card_id,
    declined,
    ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS rn
  FROM transactions
)
#con esto consigo que me empiece a contar cada vez que cambia la targeta y me ordena por fecha según la última

366 )
367
368 SELECT
369   card_id,
370   CASE
371     WHEN SUM(declined) = 3 THEN 'targeta inactiva' #Para determinar si la targeta está activa
372     ELSE 'targeta activa' # o inactiva hago un CASE
373   END AS status
374 FROM last_transactions
375 WHERE rn <= 3
376 GROUP BY card_id;
377
378 # cantidad de targetas activas

```

Figura 51. Inserción de los datos en la tabla status_cards.

Una vez creada la tabla status_cards e introducidos los datos, se establecen la PK y las Fk que permitan relacionar esta tabla con las otras tablas de dimensiones.

Se procede a determinar la **PK** de la **tabla status_cards**, el campo *card_id*, que tiene el mismo tipo de valor que en la **tabla transactions**:

```

385 • ALTER TABLE status_cards
386   ADD PRIMARY KEY (card_id);
387
388

```

Figura 52. Determinación del campo card_id como PK de la tabla status_cards.

Se establece la FK de la **tabla status_cards** que la relaciona con la **tabla credit_cards** mediante el campo *card_id*. En este caso *card_id* funcionará como PK y como FK. La **tabla status_cards** es una tabla de apoyo de la **tabla credit_cards**.

```

388 • ALTER TABLE status_cards
389   ADD CONSTRAINT fk_status_cards_credit_cards
390   FOREIGN KEY (card_id)
391   REFERENCES credit_cards(id);
392
393

```

Figura 53. Determinación de card_id como FK.

4.3 Ejercicio del Nivel 2

Ejercicio 1 – Determinar tarjetas activas/inactivas

El ejercicio consiste en que una vez creada la tabla `status_cards` y clasificadas cada tarjeta como activa o inactiva, según las últimas tres transacciones, determinar el número de tarjetas que están activas.

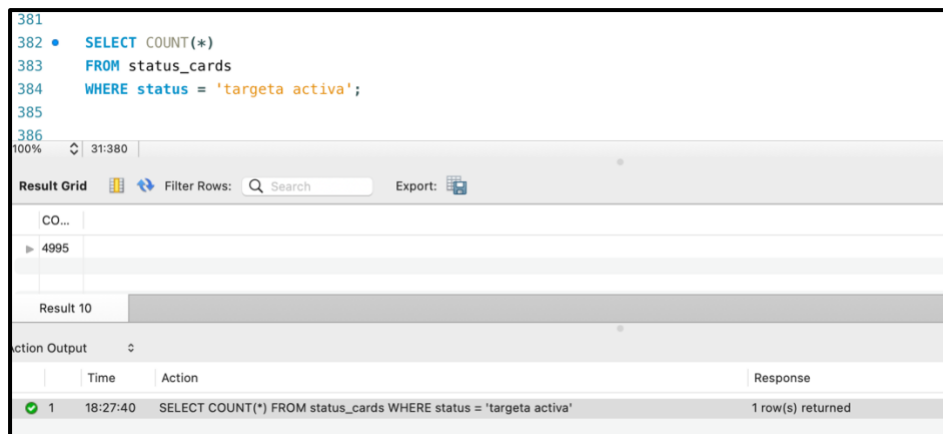


Figura 54. Número de tarjetas que están activas.

5. Nivel 3 – Relación productos–transacciones

5.1 Objetivos del Nivel 3

En el Nivel 3 se construye una tabla de transición que permita relacionar de forma adecuada los productos con las transacciones, ya que el campo `product_ids` de `transactions` contiene múltiples valores.

5.2 Creación de la tabla `products_transactions`

Se crea la **tabla `products_transactions`** para resolver la relación de muchos a muchos entre `products` y `transactions`. Cada fila representa la relación entre una transacción y uno de los productos asociados.

Debido al formato de `product_ids` se realiza una transformación previa:

1. Se genera una tabla intermedia basada en JSON para descomponer el campo `product_ids`.
2. A partir de esta tabla intermedia, se insertan los valores en la tabla `products_transactions`.

```

398 • CREATE TABLE IF NOT EXISTS products_transactions (
399     transaction_id VARCHAR(255),
400     product_id INT,
401     PRIMARY KEY (product_id, transaction_id), # en el caso de una tabla de transición la PK es la combinación de las dos
402     FOREIGN KEY (product_id) REFERENCES products(id),
403     FOREIGN KEY (transaction_id) REFERENCES transactions(id)
404 );
405
100% 5:405 1 error found

```

Action Output

	Time	Action	Response
1	17:01:44	CREATE TABLE IF NOT EXISTS products_transactions (transaction_id VARCHAR(255), product_id INT, PRIMARY KEY (product_id, transacti...	0 row(s) affected

Figura 55. Creación de la tabla products_transactions

```

406
407 • INSERT INTO products_transactions (transaction_id, product_id)
408     SELECT
409         t.id,
410         jt.product_id
411     FROM transactions t
412     JOIN JSON_TABLE(
413         CONCAT('[', t.product_ids, ']'), #convierte "3,5,8" en "[3,5,8]"
414         "$[*]" COLUMNS(
415             product_id INT PATH "$" #cada elemento del array como fila
416         )
417     ) AS jt;
418
100% 9:417 1 error found

```

Action Output

	Time	Action	Response
1	17:02:44	INSERT INTO products_transactions (transaction_id, product_id) SELECT t.id, jt.product_id FROM transactions t JOIN JSON_TABLE(CON...	253391 row(s) affected Records: 253391
2	17:04:15	SELECT * FROM new_transactions.products_transactions	253391 row(s) returned

Figura 56. Inserción de los datos en la tabla products_transactions mediante la creación de una tabla JSON.

5.3 Ejercicio del Nivel 3

Ejercicio 1 – Número de ventas por producto

Se desarrolla una consulta para calcular cuántas veces se ha vendido cada producto.

```

SELECT pt.product_id, p.product_name, COUNT(pt.transaction_id) AS num_ventas
FROM products_transactions pt
JOIN transactions t ON t.id = pt.transaction_id
JOIN products p ON p.id = pt.product_id
WHERE t.declined = 0
GROUP BY p.product_name, pt.product_id
ORDER BY num_ventas DESC;

```

Result Grid

product_id	product_name	num_ventas
52	riverlands the duel	2642
29	Tully maester Tully	2627
21	duel Direwolf	2603
16	the duel warden	2602
33	duel warden	2593
87	sith Jade	2591
66	mustafar jinn	2590
48	rock Renly in	2589
68	Stark Karstark	2587
23	riverlands north	2586
88	Stannis warden s...	2582
73	Dorne bastard	2579
34	of north	2574
4	warden south duel	2573

Action Output

	Time	Action	Response
1	17:26:46	SELECT pt.product_id, p.product_name, COUNT(pt.transaction_id) AS num_ventas FROM products_transactions pt JOIN transactions t ON t.id = ...	100 row(s) returned

Figura 57. Consulta para saber cuántas veces se ha vendido cada producto.

6. Diagrama final del modelo

Tras completar los tres niveles, el diagrama final refleja:

- **transactions** como tabla de hechos central.
- **users**, **companies**, **credit_cards**, **products** como tablas de dimensiones y **status_cards** como tabla de apoyo de la **tabla credit_cards**.
- **products_transactions** como tabla de transición para solucionar la relación de muchos a muchos entre productos y transacciones. Esta tabla permite convertir una relación de M:N a dos relaciones de 1:N. Esto permite almacenar una fila por combinación **transaction_id**–**product_id**, evita anomalías de datos, como duplicidades, y posibilita cálculos correctos de métricas como número de ventas por producto o importe total asociado a cada producto.

El diagrama final una vez realizados los 3 niveles es el siguiente:

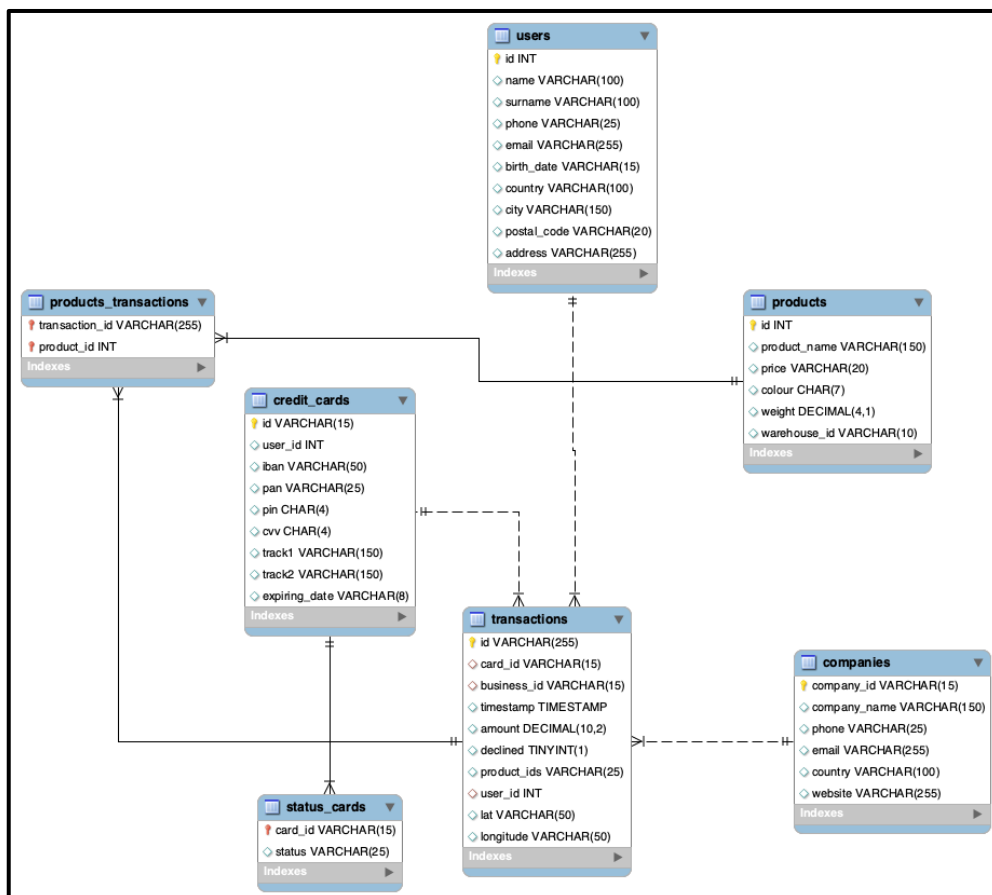


Figura 58. Diagrama final.

7. Conclusiones

La tarea ha permitido diseñar y construir una base de datos siguiendo un modelo en estrella, cargar y transformar datos desde ficheros CSV, y aplicar técnicas avanzadas de SQL (funciones de ventana, CTE, tablas derivadas y tablas de transición) para resolver requisitos analíticos específicos.

El modelo resultante mantiene un esquema en estrella, con **transactions** como tabla de hechos central. Además de las dimensiones principales (**users**, **companies**, **credit_cards**, **products**), incluye una tabla derivada que sería la tabla **status_cards** y una tabla de transición **products_transactions**. Esta tabla permite resolver la relación muchos-a-muchos que existe entre la tablas tablas **products** y **transactions**.

El resultado es un modelo preparado para responder preguntas de negocio sobre usuarios, empresas, tarjetas, productos y transacciones, incluyendo análisis de estados de tarjetas y volumen de ventas por producto.